

SynthOmnicon — Lean 4 Formalization

March 25, 2026

Lean 4 / Mathlib formalizations connected to the SynthOmnicon framework. Toolchain: **Lean 4.28.0** · **Mathlib v4.28.0**

0.1 Library structure

```
SynthOmnicon/  
Primitives/  
  Core.lean          -- 12 primitive types with lattice structure + cross-  
    primitive axioms  
  OPN_2adic.lean     -- Classical number theory: 2-adic valuation +  
    Touchard congruence for OPNs
```

0.2 OPN_2adic.lean — What is actually proven

This file formalizes the **2-adic and 3-adic valuation arguments** constraining the structure of odd perfect numbers (OPNs), culminating in a machine-verified proof of **Touchard’s congruence (1953)**. Every **sorry** is an honest marker of a genuine open problem; no result is overclaimed.

0.2.1 Definitions

Symbol	Definition
Perfect n	$\text{sigma}(n) = 2n$, using Mathlib’s <code>ArithmeticFunction.sigma 1</code>
$v\ n$	$(\text{Nat.factorization } n)\ 2$ — 2-adic valuation via prime factorization

0.2.2 Helper lemmas — fully proved, no sorry

2-adic modular arithmetic

σ identities and product formula

3-adic modular arithmetic (for Touchard)

0.2.3 Main theorems

euler_opn_form (*sorry* — Euler 1747, not yet in Mathlib)

Every OPN has the form $n = p^k \cdot m^2$ with p prime, $p \equiv k \equiv 1 \pmod{4}$, $p \nmid m$.

opn_product_constraint *fully proved*

Lemma	Statement
pred_dvd_pow_sub_one	$(p-1) \mid (p^n - 1)$ — via <code>geom_sum_mul</code> over $\mathbb{Z}$
v2_div_of_dvd	$v_2(a/b) = v_2(a) - v_2(b)$ when $b \mid a$
v2_eq_one_of_mod4_eq2	$n\%4 = 2 \rightarrow v_2(n) = 1$ — via <code>Prime.pow_dvd_iff_le_factorization</code>
pow_odd_of_odd	$q\%2 = 1 \rightarrow q^i\%2 = 1$
pow_mod4_of_mod4	$p\%4 = 1 \rightarrow p^i\%4 = 1$
geom_sum_mod2	$(\sum_{i<n} q^i)\%2 = n\%2$ when q is odd
geom_sum_odd_mod2	$(\sum_{i<2e+1} q^i)\%2 = 1$ — odd count of odd terms is odd
geom_sum_mod4	$(\sum_{i<n} p^i)\%4 = n\%4$ when $p \equiv 1 \pmod{4}$
geom_sum_mod4_eq2	Corollary: $n\%4 = 2 \rightarrow (\sum_{i<n} p^i)\%4 = 2$

Lemma	Statement
sigma_mul_of_coprime	$\sigma(ab) = \sigma(a)\sigma(b)$ for $\gcd(a, b) = 1$ — from <code>isMultiplicative_sigma</code>
sigma_prime_pow_ratio	$\sigma(p^k) \cdot (p-1) + 1 = p^{k+1}$ — geometric sum identity in $\mathbb{N}$
sigma_prime_pow_lt	$\sigma(p^k) \cdot (p-1) < p^{k+1}$ — the ratio $\sigma(p^k)/p^k$ never reaches the ceiling $p/(p-1)$
opn_mod4	Any $\text{OPN} \equiv 1 \pmod{4}$ — from $p \equiv 1 \pmod{4}$ and m odd $\rightarrow m^2 \equiv 1 \pmod{4}$

Lemma	Statement
pow_mod3_q2_even	$q\%3 = 2 \rightarrow q^{2e}\%3 = 1$ — since $q^2 \equiv 1 \pmod{3}$
pow_mod3_q2_odd	$q\%3 = 2 \rightarrow q^{2e+1}\%3 = 2$
geom_sum_mod3_q2	$(\sum_{i<n} q^i)\%3 = n\%2$ when $q \equiv 2 \pmod{3}$ — pairs cancel mod 3
pow_mod3_one	$p\%3 = 1 \rightarrow p^k\%3 = 1$
sq_mod3_of_not_dvd	$\neg 3 \mid m \rightarrow m^2\%3 = 1$
sigma_dvd3_of_p2_kodd	$p\%3 = 2, k\%2 = 1 \rightarrow 3 \mid \sigma(p^k)$

`sigma_prime_pow_ratio` *fully proved*

`v2_sigma_prime_power` *fully proved*

For odd prime $p \equiv 1 \pmod{4}$ and $k \equiv 1 \pmod{4}$: $v_2(\sigma(p^k)) = 1$

Proof chain: $\sigma(p^k) = \sum_{i \leq k} p^i$ (via `Finset.sum_map` after `Nat.divisors_prime_pow`) $\rightarrow$ sum $\equiv k + 1 \equiv 2 \pmod{4}$ (no LTE needed) $\rightarrow v_2 = 1$.

`v2_sigma_square_factor` *fully proved*

For odd prime q and any e : $v_2(\sigma(q^{2e})) = 0$

Proof chain: $\sigma(q^{2e}) = \sum_{i \leq 2e} q^i \rightarrow$ sum of $2e + 1$ odd terms is odd $\rightarrow v_2 = 0$.

`v2_accumulation_constraint` *fully proved*

Given the Euler decomposition $n = p^k \cdot m^2$: $v_2(\sigma(p^k)) = 1$ and $\forall q \mid m, \exists e, v_2(\sigma(q^{2e})) = 0$

Machine-verified statement that the 2-adic constraint is **necessary** for any OPN. Does not prove nonexistence — it characterizes what any OPN must look like.

`opn_mod4` *fully proved*

`touchard_congruence` *fully proved*

`opn_nonexistence` (*sorry — open problem*)

0.3 Core.lean — Primitive type system

Defines the 12 `SynthOmnicon` primitives as inductive types with deriving `DecidableEq`, `Repr`, `Ord`, plus explicit LE instances and four cross-primitive axioms:

Axiom	Content
A	$H_\infty \rightarrow K_{\text{trap}}$ (topological chirality implies kinetic trapping)
B	$\Omega \geq \Omega_Z \rightarrow H \geq H_2$ (integer winding number requires persistent chirality)
C	$D_{\text{holo}} \leftrightarrow T_{\text{holo}}$ (holographic dimensionality iff holographic topology)
D	$\Omega_{\text{NA}} \rightarrow D_{\text{holo}}$ (non-Abelian anyonic protection requires holographic substrate)

0.4 Build

```
lake build SynthOmnicon
```

Expected output: Build completed successfully with warnings only (two honest `sorry` markers, three unused variable hints from over-specified hypotheses).

0.5 Proof-engineering notes

Several Lean 4.28.0 / Mathlib API subtleties encountered and resolved:

- `Nat.divisors_prime_pow` returns `Finset.map` (with a `Function.Embedding`), **not** `Finset.image`. Use `Finset.sum_map` to unfold, not `Finset.sum_image`.
- `omega` cannot cross `Finset.sum` barriers. Fix: introduce intermediate modular arithmetic steps that let `omega` work on plain `N` expressions.
- The induction hypothesis in a `suffices h : P n by ...` proof carries `P n`'s own hypotheses into the IH. Factor out the unconditional general lemma first, then apply it with the specific hypothesis.
- `norm_num` primality extension requires `import Mathlib.Tactic` (not just targeted imports).
- `rw [pow_one]` fails inside a `Finset.sum` after certain rewrite sequences; `simp` after `Finset.sum_map` handles the residual $(p)^1 = p$ correctly.
- Numeric `rw` chains like `rw [Nat.mul_mod, hq]` can fully close goals via `rfl` when the resulting expression is a closed numeral (e.g. $2 * 2 \% 3$). Appending `norm_num` after such a chain produces a "no goals" error — omit it when the `rw` already closes.
- `absurd h hp3` fails when $h : 3 = p$ but $hp3 : p \neq 3$ (wrong `Ne` orientation). Use `omega` or `Ne.symm` to bridge the gap.
- `Dvd.dvd.mul_left` does not exist in this Mathlib version. Use `dvd_mul_of_dvd_right (dvd_pow h hn) _` instead.

0.6 Design notes and Q&A

0.6.1 Relationship between `Core.lean` and `OPN_2adic.lean`

`OPN_2adic.lean` does not import `Core.lean`, and at present there is no Lean-level integration between the two files. The connection is **conceptual and meta-level**: the SynthOmnicon primitive framework is used as a thinking tool to organize and motivate the number-theoretic argument, not as formal input to the proofs.

The specific framing that guided this work: any OPN must resemble a molecule with a unique "functional group" (the Euler prime p^k , carrying exactly one unit of 2-adic charge) bonded to an inert "molecular scaffold" (the square factor m^2 , contributing zero to the 2-adic budget). The global "stoichiometric constraint" $\sigma(n) = 2n$ together with the neutrality condition n odd then severely overdetermines the system. This is the SynthOmnicon constraint-propagation picture applied to number theory. The primitives that map most naturally onto this picture are:

OPN concept	SynthOmnicon analogue
Euler prime p^k — unique 2-adic carrier	Φ_c — absorbing under meet, unique criticality carrier
Square factors q^{2e} — 2-adically inert	Φ_{sub} with K_{trap} isolation
$\sigma(n) = 2n$ — global balance	S (Stoichiometry) — exact ratio constraint
n odd — parity conservation	P (Polarity) — parity / neutrality condition

The phrase "mapping down from a more fundamental constraint space" refers to this conceptual translation, not to any automated or formal machinery. No custom tactic, external solver, or category-theoretic functor is involved. The mapping is a heuristic that shaped *which* lemmas to prove and in what order; correctness is verified entirely by Lean in the usual way.

A formal integration — where OPN variables are assigned primitive tuples and the constraint propagation is machine-checked at the SynthOmnicon level — is a long-term goal but is not yet planned for Lean.

0.6.2 Cross-primitive axioms in `Core.lean`

The four `axiom` declarations are **foundational postulates** of the SynthOmnicon framework, not goals to be derived from the inductive type definitions. Within the current Lean representation — where each primitive is just a finite inductive type with `Ord` — there is no path to deriving, say, `D_holo_iff_T_holo`

from first principles, because the types carry no algebraic structure that encodes the physical relationship between dimensionality and topology. Deriving such equivalences would require an external interpretation: a functor from the primitive lattice into a category where boundary/bulk duality has an intrinsic meaning (e.g., a topos-theoretic or operadic model). That is out of scope for the current formalization.

On LE instances vs LinearOrder: the custom LE via `compare` is intentionally minimal. The types that need it (`Protection`, `Chirality`) only use $\geq$ in the cross-primitive axioms, not lattice operations. Deriving a full `LinearOrder` is straightforward — all types derive `Ord` and the machinery exists in `Mathlib` — but premature: it would be needed only once $\sqcap/\sqcup$ operations are actually used in proofs. The exception is `Criticality`, where `Phi_c` is *absorbing* under meet (`meet(Phi_c, x) = Phi_c` for all `x`), which contradicts `min` semantics. A correct `MeetSemilattice` instance for `Criticality` will require a custom definition that overrides the `Ord`-derived ordering for meet — this is noted in a comment in `Core.lean` and is the first planned extension to the primitive lattice.

0.6.3 OPN formalization details

Coprimality hypothesis in touchard_congruence. The `hcop : Nat.Coprime (p^k) (m^2)` in `touchard_congruence` (and in `opn_product_constraint`) is an explicit parameter rather than something derived, because `euler_opn_form` is still sorry. Once `euler_opn_form` is proved, `hcop` becomes derivable: `p prime, p m → Nat.Coprime p m → Nat.Coprime (p^k) (m^2)` by `Nat.Coprime.pow`. At that point, `hcop` can be dropped from the signature and threaded through internally. The current signature is a faithful statement of what the proof actually uses, without hiding a dependency on the sorry.

factorization.support vs forall q, q.Prime -> q m. The `factorization.support` indexing is intentional: it provides the exponent `(Nat.factorization m) q` directly, which is exactly the `e` needed for the existential $\exists e, v_2(\sigma(q^{2^e})) = 0$. An alternative phrasing with `q.Prime -> q m` would require a separate argument that the correct exponent exists and is given by the factorization — more work for no gain in readability.

euler_opn_form in Mathlib. As of Lean 4.28.0 / `Mathlib` v4.28.0, the Euler form for OPNs ($n = p^k m^2, p \equiv k \equiv 1 \pmod{4}$) is **not in Mathlib**. The number theory coverage in `Mathlib` is extensive for multiplicative functions, divisor sums, and primality, but this specific structural theorem — which requires a careful argument about the 2-adic structure of $\sigma(n) = 2n$ applied globally across the full factorization — has not been formalized. Proving it from the tools already in this file is feasible: the key lemmas (`sigma_mul_of_coprime`, `sigma_prime_pow_ratio`, `v2_eq_one_of_mod4_eq2`) are all in place. It is a planned extension, not a fundamental obstacle.

0.6.4 Other Mathlib API surprises

Beyond the items already in the proof-engineering notes:

- **zify is required to use geom_sum_mul.** The `Mathlib` `geom_sum_mul` lemma operates over a `CommRing`, so it lives in $\mathbb{Z}$ (or any ring), not $\mathbb{N}$. The `zify` tactic — with guards `[hp.one_le, Nat.one_le_pow ...]` to handle the coercion side conditions — is the cleanest bridge. Attempting to prove the geometric identity directly in $\mathbb{N}$ avoids the sign issues but requires manual subtraction accounting.
- **v2_eq_one_of_mod4_eq2 design choice.** The `by_contra / omega` approach was preferred over a more direct use of `Prime.pow_dvd_iff_le_factorization` because the contrapositive direction ($\text{factorization} \geq 2 \rightarrow 2^2 \mid n$) is what the lemma naturally produces, and making both the lower and upper bound explicit as separate steps keeps the proof readable. A more compressed proof using `Nat.le_antisymm` with two applications of `pow_dvd_iff_le_factorization` is possible and would be slightly shorter; the current form was chosen for clarity.
- **Unused variable warnings in v2_accumulation_constraint (h_perf) and v2_sigma_prime_power (hp_odd)** are intentional: those hypotheses are included because they are part of the natural statement of the theorem (an odd perfect number has an odd prime in its Euler form) even though the specific proof path doesn't reach them. They document the full precondition rather than minimizing the signature. They can be removed without affecting correctness.

0.6.5 The "mapping down" claim — honest account

There is no automated reasoning, custom tactic, or external solver involved. The phrase "mapping down from a more fundamental constraint space" refers to using the SynthOmnicon conceptual framework — specifically its vocabulary of unique constraint carriers, neutral scaffolds, and overdetermined global balance equations — as a *problem-structuring heuristic*. This heuristic suggested:

1. Look for the unique carrier of the conserved quantity (v_2 budget = 1, carried by p^k).
2. Verify the scaffold is transparent (square factors contribute zero).
3. Write down the overdetermined global constraint and check consistency.
4. Apply the same template at the next prime (3-adic version for Touchard).

All proofs are standard Lean 4 / Mathlib proofs verified by the kernel. The framework provided direction, not machinery.

0.6.6 Future directions

Connecting the two tracks. The most concrete near-term connection: as `Core.lean` gains `Semilattice` / `Lattice` instances, the OPN constraint structure could be encoded as a *synthon* — a tuple in the 12-dimensional primitive space — and the constraint propagation verified at that level. The Φ_c absorbing-meet property is the structural analogue of the Euler prime's uniqueness; formalizing this analogy in Lean would be the first real integration between the files. This is planned but not yet scoped.

Lattice instances for `Core.lean`. The immediate next step for `Core.lean` is implementing the custom `MeetSemilattice` for `Criticality` (where Φ_c is absorbing). After that, full `Lattice` instances for the five ordered primitives (F, K, G, Ω, H) are straightforward using `minFac`/`maxFac` over the `Ord`-derived linear order. The categorical primitives (D, T, R, P, Γ) require a different treatment: their meet semantics are identity-or-bottom, not min, so they need a custom `BoundedLattice` with $\perp$ representing "incompatible".

OPN track. The natural next steps in order of difficulty:

1. Prove `euler_opn_form` (all required tools are present).
2. Extend to `touchard_congruence_v2` without the Euler decomposition as a hypothesis (i.e., derive the decomposition inside the proof from `euler_opn_form`).
3. Begin the prime bound argument: from `opn_product_constraint` and `sigma_prime_pow_ratio`, derive a lower bound on the number of distinct prime factors of any OPN.